

Distributed Sampling Strategies for Class Imbalance in Big Data: A PySpark Study on Financial Fraud Detection

Muthukamaran Nallusamy, Rutuj Bafna, Tharun Gurunathan
School of Computer Science, University of Nottingham, United Kingdom

Abstract—Class imbalance significantly impedes fraud detection, where the PaySim dataset contains (6,362,620 instances) with only 8,213 instances being fraudulent for a class imbalance ratio of 769:1. Classifiers trained on datasets without any sampling methods will classify all instances as belonging to the majority class, resulting in no detection of fraud cases. Four distributed sampling methods were applied, namely random undersampling (S-01), distributed SMOTE (S-02), SMOTE+Tomek Links (S-03), and EasyEnsemble (S-04). These sampling methods are native implementations within PySpark running on a four-worker Databricks cluster. Two algorithms were then evaluated, which include Random Forest (RF) and Logistic Regression (LR) models, where the data was scaled by 10%, 30%, and 100%. At full data scale, RF with SMOTE produces the best results with Recall 0.729, Precision 0.972, F1 0.833. The application of SMOTE on the LR algorithm produces very marginal gains compared to RF. Therefore, a linear decision boundary cannot benefit from the non-linear synthetic minority manifold.

Keywords—*Apache Spark, PySpark, class imbalance, distributed SMOTE, EasyEnsemble, Tomek Links, BucketedRandomProjectionLSH, RDD mapPartitions, PaySim, fraud detection, Random Forest, Logistic Regression, Databricks.*

I. INTRODUCTION

The PaySim dataset [6] simulates one month of real mobile money transactions across 6,362,620 rows, of which only 0.13% (8,213 records) are fraudulent—a 769:1 class imbalance. When a standard classifier is trained on such skewed data it learns that predicting “legitimate” for every transaction yields 99.87% accuracy yet catches zero fraud. Our S-00 baseline confirms this: Logistic Regression Fraud Recall = 0.000 at all three scales; even Random Forest achieves only 0.374 Recall at 10% scale.

Resampling methods deal with class imbalance using undersampling, where majority class examples are discarded, or oversampling, where artificial examples of the minority class are created. Methods like SMOTE [1], SMOTE + Tomek [4], and EasyEnsemble [3] are well-known in sequential machine learning applications. Yet, these methods were developed for data sets that can be stored in a single machine’s memory. Naive application of these algorithms on 6.3 million transactions, which is distributed across worker nodes, results in serious inefficiencies: any operation that needs a global perspective causes a costly shuffle.

The paper contributes to big-data research in three major ways: (1) partition-based parallel SMOTE with RDD.mapPartitions and broadcast KNN to synthesize data in-memory on worker nodes without shuffle; (2) parallel Tomek Links filtering with BucketedRandomProjectionLSH approximate nearest-neighbour join to overcome $O(n^2)$ complexity impossible in millions of records; (3) 30-condition benchmarking test (5 resampling methods $\times$ 2 classification models $\times$ 3 data sizes) highlighting the interplay between classifier geometry and resampling efficiency at big data volume.

The research questions that will be covered are: (RQ1) Which distributed sampling technique offers an optimum balance between precision and recall for detecting fraud? The research questions to be covered include the following: (RQ1) What is the most efficient distributed sampling technique that offers an optimum balance between precision and recall for fraud detection? (RQ2) What role does classification play in the efficiency of distributed sampling techniques? (RQ3) How do the different sampling techniques scale up to 10%, 30%, and 100% of the PaySim dataset?

II. RELATED WORK

Chawla et al. [1] introduced SMOTE, generating minority samples by linear interpolation between a sample and its k-nearest neighbours in feature space. Tomek [2] proposed Tomek Links—opposite-class pairs that are mutual nearest neighbours—whose majority-side removal cleans the decision boundary. Batista et al. [4] combined SMOTE and Tomek Links into a two-phase hybrid. He and Garcia [9] establish recall-focused evaluation as the standard for skewed datasets.

Liu et al. [3] proposed EasyEnsemble: training multiple base learners each on all minority samples plus a

random majority subsample, then aggregating predictions by majority vote. Fernandez et al. [5] survey big-data imbalance learning, noting that most Sparkbased approaches apply oversampling before data distribution—the opposite of the partition-local design taken here. Datar et al. [10] introduce Locality-Sensitive Hashing (LSH), which underpins Spark MLlib’s BucketedRandomProjectionLSH used in our Tomek cleaning step.

III. METHODOLOGY

A. Data Loading and Feature Engineering

PaySim Data is read using `spark.read.csv (header=True, inferSchema=True)` in CSV format. The process of reading takes place in parallel because the file is split up among the worker nodes, and each executor reads consecutive data from DBFS, forming a narrow transformation that does not require any shuffling. The `transaction_type` field is mapped to integer values using the following mapping: `TRANSFER=1, CASH_OUT=2, PAYMENT=3, CASH_IN=4`, otherwise 0.

`VectorAssembler` merges seven attributes into `raw_features` vector. The `StandardScaler (withMean=True, withStd=True)` normalises in a distributed two-pass computation wherein the first pass computes the means and stds per feature while the second pass performs the z-score normalisation of each row locally. Normalisation is a necessary step for the application of SMOTE since without normalisation, the balance features (which contain very large values such as 10 million+) would dominate the Euclidean distance metric and lead to a non-functioning KNN algorithm.

B. Distributed Sampling Strategies S-00 Baseline: No resampling. Raw imbalanced data is passed directly to the classifier. Serves as the lower bound, demonstrating the severity of the 769:1 imbalance problem.

S-01 Random Undersampling: The majority class is sampled without replacement to achieve a 2:1 ratio. `Fraction = min (fraud×2 / legit, 1.0)`, `seed=42`. Applied via `df.sample ()`, a narrow transformation: each partition independently keeps or drops rows, with no shuffle. Fastest strategy but discards 99.7% of legitimate transaction data at full scale.

S-02 Distributed SMOTE: Standard SMOTE is sequential and memory-bound ($O(n^2)$ distance matrix). Our distributed implementation avoids both limitations. The minority class (8,213 rows at full scale, ≈460 KB) is collected to the driver and broadcast via `SparkContext.broadcast()`—safe at this size and avoids repeated serialisation. An RDD of minority indices is parallelised across 4 worker nodes via `parallelize(range(n_fraud), numSlices=4)`. Each partition runs `mapPartitions`, reading from the broadcast variable (already in executor memory, no network hop), computing KNN distances with NumPy, and generating synthetic samples by linear interpolation. The result is unioned with the original training data—`union()` is a narrow transformation requiring no shuffle. The oversample multiplier is capped at 50× to prevent memory overflow at 100% scale.

S-03 SMOTE + Tomek Links: Tomek Links cleaning removes majority-side samples from opposite-class

mutual nearest-neighbour pairs, sharpening the decision boundary. Sequential implementations require $O(n^2)$ pairwise distance computation—infeasible at 6M+ rows. We replace this with

`BucketedRandomProjectionLSH (bucketLength=20.0, numHashTables=3, threshold=2.0)` from Spark MLlib, reducing complexity to approximately $O(n \log n)$. A 0.2% majority subsample joins all minority rows via `approxSimilarityJoin`; identified majority Tomek members are removed with a left-anti join. Phase 1 runs S-02 SMOTE (cached); Phase 2 runs LSH cleaning. The two-phase design means S-03 inherits S-02's recall improvement and adds precision improvement from boundary cleaning.

S-04 EasyEnsemble: Five RF sub-learners are each trained on all minority samples plus a random majority subsample (`fraction = fraud/legit`, `seed = 42+i` for diversity). Predictions are aggregated by averaging sublearner scores and thresholding at 0.5. Unlike SMOTE-based strategies, no synthetic data is generated—the ensemble exploits different random majority views to overcome imbalance.

C. Classifiers

Random Forest [8] (`numTrees=10`, `maxDepth=5`, `seed=42`) is used as the primary classifier. MLlib builds trees in parallel across executors; RF's recursive axis-aligned partitioning captures the irregular non-linear minority manifold produced by SMOTE. Logistic Regression (`maxIter=20`, `regParam=0.01`, `elasticNetParam=0.0`) provides a linear baseline. Its single hyperplane cannot capture the non-convex SMOTE manifold, making it a structural negative control for RQ2.

D. Evaluation Protocol

There are three scales analyzed: 10% (636,780 instances, 822 frauds), 30% (1,909,895 instances, 2,504 frauds), and 100% (6,362,620 instances, 8,213 frauds). `stratified.sampleBy` maintains the ratio of 769:1 for all scales. `randomSplit([0.8, 0.2], seed=42)` is used to perform the train/test split; all transformations related to the sampling process are done on the training dataset only. There are four performance measures: Fraud Recall ($TP/(TP + FN)$, key measure), Fraud Precision ($TP/(TP + FP)$), F1 score (harmonic mean of recall and precision)

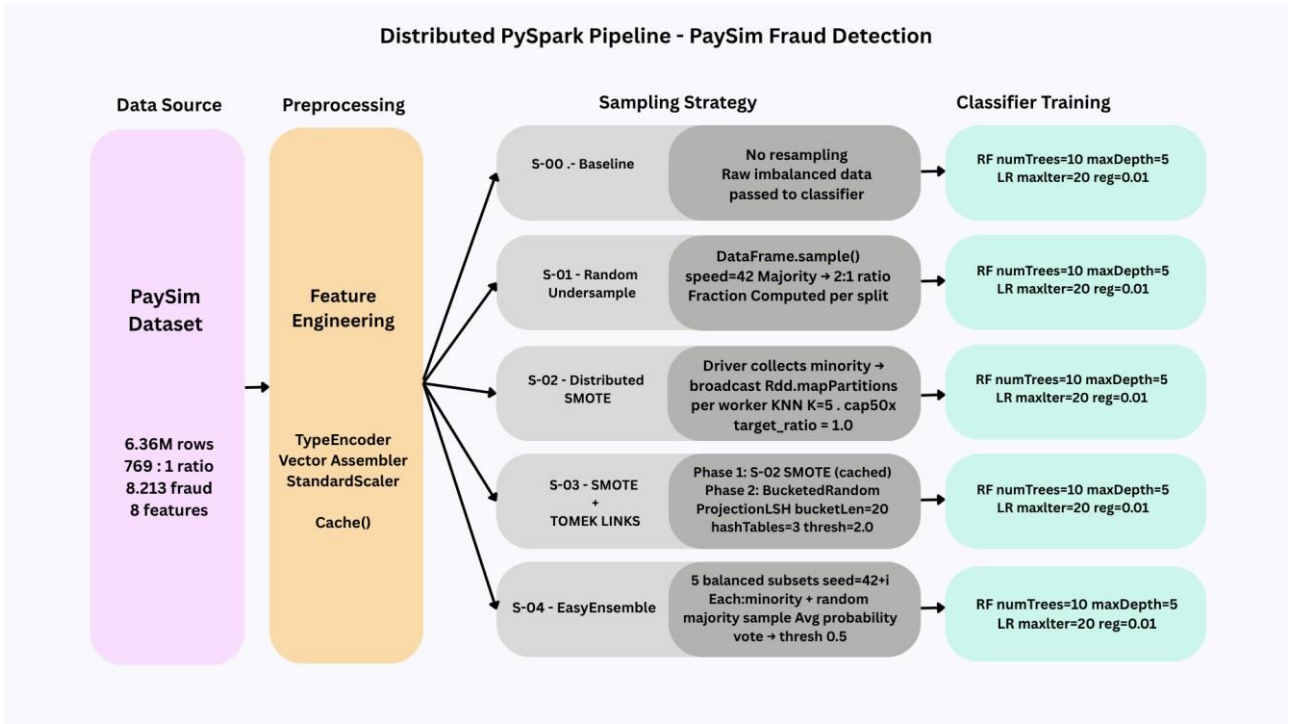


Fig. 1. End-to-end distributed pipeline on Databricks (4 workers). PaySim CSV → feature engineering → StandardScaler → stratified split → sampling strategy per partition (RDD.mapPartitions for SMOTE, LSH for Tomek) → RF or LR training → evaluation on driver.

IV. EXPERIMENTAL STUDY AND RESULTS

Tables I–VI report results across all 30 experimental conditions (5 strategies × 2 classifiers × 3 scales). Highlighted rows indicate the best F1 score within each classifier-scale combination. Bold F1 values identify the recommended strategy per condition.

A. 10% Scale (636,780 rows | ~822 fraud)

S-01 and S-04 achieve near-perfect RF Recall (0.981) but Precision collapses to ~2%, yielding F1 scores of only 0.041 and 0.051 respectively—operationally undeployable. S-03 SMOTE+Tomek achieves the best F1 at this scale (0.772) by balancing Recall 0.665 with Precision 0.920. The RF-LR SMOTE Recall gap is 4.3× (0.710 vs 0.165), confirming that LR’s linear boundary cannot exploit the non-linear synthetic manifold.

TABLE I. Random Forest — 10% Scale. Highlighted = best F1.

Strategy	Recall	Precision	F1 Score	AUC-ROC	Time (s)
S-00 Baseline	0.374	1.000	0.544	0.950	8.15
S-01 Undersamp.	0.981	0.021	0.041	0.994	15.10
S-02 SMOTE	0.710	0.539	0.613	0.991	24.61
S-03 SMOTE+Tomek	0.665	0.920	0.772	0.989	49.56
S-04 EasyEnsemble	0.981	0.026	0.051	0.994	103.18

TABLE II. Logistic Regression — 10% Scale.

Strategy	Recall	Precision	F1 Score	AUC-ROC	Time (s)
S-00 Baseline	0.000	0.000	0.000	0.898	41.93
S-01 Undersamp.	0.867	0.008	0.016	0.914	28.64
S-02 SMOTE	0.165	0.092	0.118	0.904	83.50
S-03 SMOTE+Tomek	0.165	0.092	0.118	0.904	116.31
S-04 EasyEnsemble	0.861	0.008	0.016	0.914	122.81

B. 30% Scale (1,909,895 rows | ~2,504 fraud)

RF The value of Baseline Recall is raised to 0.587 (F1 = 0.735). Thus, one can see that in spite of the issue that had occurred, the quantity of data points by itself manages to solve it partially. As for the recall value (0.723), both algorithms SMOTE+Tomek and SMOTE generate the same output but increase the Precision to 0.922 (F1 = 0.779 → 0.810). LR S-02 and S-03 are identical.

TABLE III. Random Forest — 30% Scale. Highlighted = best F1.

Strategy	Recall	Precision	F1 Score	AUC-ROC	Time (s)
S-00 Baseline	0.587	0.984	0.735	0.950	214.18
S-01 Undersamp.	0.987	0.029	0.056	0.996	82.38

S-02 SMOTE	0.723	0.845	0.779	0.988	101.70
S-03 SMOTE+Tomek	0.723	0.922	0.810	0.982	359.63
S-04 EasyEnsemble	0.987	0.028	0.054	0.996	209.36

TABLE IV. Logistic Regression — 30% Scale.

Strategy	Recall	Precision	F1 Score	AUC-ROC	Time (s)
S-00 Baseline	0.000	0.000	0.000	0.911	29.72
S-01 Undersamp.	0.829	0.008	0.016	0.919	28.49
S-02 SMOTE	0.209	0.126	0.157	0.918	61.17
S-03 SMOTE+Tomek	0.209	0.125	0.156	0.918	158.63
S-04 EasyEnsemble	0.827	0.008	0.016	0.919	122.10

C. 100% Scale (6,362,620 rows | 8,213 fraud) RF On a full-scale basis, SMOTE performs best with Recall 0.729, Precision 0.972, and F1 score 0.833. While SMOTE+Tomek produces comparable performance in terms of F1 score 0.831, it requires 8× more processing power (3,338 seconds versus 413 seconds) on account of the additional cost involved in the LSH join operation with respect to the augmented dataset after performing SMOTE. The EasyEnsemble approach provides the highest recall of 0.981 while failing to produce an acceptable precision of only 0.027 (F1 = 0.053).

TABLE V. Random Forest — 100% Scale. Highlighted = best F1.

Strategy	Recall	Precision	F1 Score	AUC-ROC	Time (s)
S-00 Baseline	0.640	0.991	0.778	0.962	93.22
S-01 Undersamp.	0.977	0.028	0.054	0.996	50.38
S-02 SMOTE	0.729	0.972	0.833	0.986	412.53
S-03 SMOTE+Tomek	0.727	0.970	0.831	0.989	3338.42
S-04 EasyEnsemble	0.981	0.027	0.053	0.996	411.02

TABLE VI. Logistic Regression — 100% Scale.

Strategy	Recall	Precision	F1 Score	AUC-ROC	Time (s)
S-00 Baseline	0.000	0.000	0.000	0.905	52.19
S-01 Undersamp.	0.834	0.008	0.016	0.920	31.58
S-02 SMOTE	0.239	0.114	0.154	0.918	332.96

S-03 SMOTE+Tomek	0.238	0.115	0.155	0.918	2379.48
S-04 EasyEnsemble	0.840	0.008	0.016	0.923	251.02

V. DISCUSSION

A. RQ1: Best Strategy for Fraud Detection at Scale RF SMOTE (S-02) is the recommended production strategy: Recall 0.729, Precision 0.972, F1 0.833 at 100% scale in 413 s. SMOTE+Tomek (F1 0.831) is statistically indistinguishable from plain SMOTE at full scale, making its 8× runtime overhead unjustifiable.

B. RQ2: Classifier-Strategy Interaction The 4.3× RF-LR Recall gap on SMOTE (0.710 vs 0.165 at 10% scale, narrowing to 3.1× at 100%) is the central finding. SMOTE generates synthetic points by linear interpolation within the minority manifold—an irregular, non-convex region shaped by real fraud feature distributions. RF’s recursive axis-aligned splits can carve out this region with fine-grained boundaries. LR’s single hyperplane cannot; applying SMOTE with a linear classifier wastes sampling computation and may increase false positive rates without capturing true minority structure.

Tomek cleaning is likewise ineffective for LR at all scales: a linear boundary cannot be sharpened by removing boundary samples if the boundary’s shape is architecturally constrained. These results establish that the choice of classifier must precede the choice of sampling strategy in system design.

C. RQ3: Scalability

RF training time is approximately linear with data volume for S-00 and S-01. S-02 SMOTE adds constant broadcast overhead plus linear mapPartitions synthesis, growing modestly from 24.61 s (10%) to 412.53 s (100%). S-01 counterintuitively decreases in relative training time as scale grows because undersampling keeps the balanced training set small (~16,000 rows) regardless of total dataset size. S-03 Tomek reaches 3,338 s at 100% scale: the LSH approxSimilarityJoin over the post-SMOTE enlarged dataset grows steeply, representing a scalability bottleneck that would worsen on larger datasets. S-04 EasyEnsemble runtime (~411 s at 100%) reflects five sequential RF fits, each on a small balanced subset.

A limitation of this study is that the four-worker Databricks cluster limits the degree of parallelism measurable. On a larger cluster (16+ workers) the broadcast-based SMOTE design would scale linearly, while S-03’s LSH join overhead would remain a bottleneck due to the wide transformation it requires. Future work should evaluate S-02 on a larger cluster to confirm linear scalability empirically.

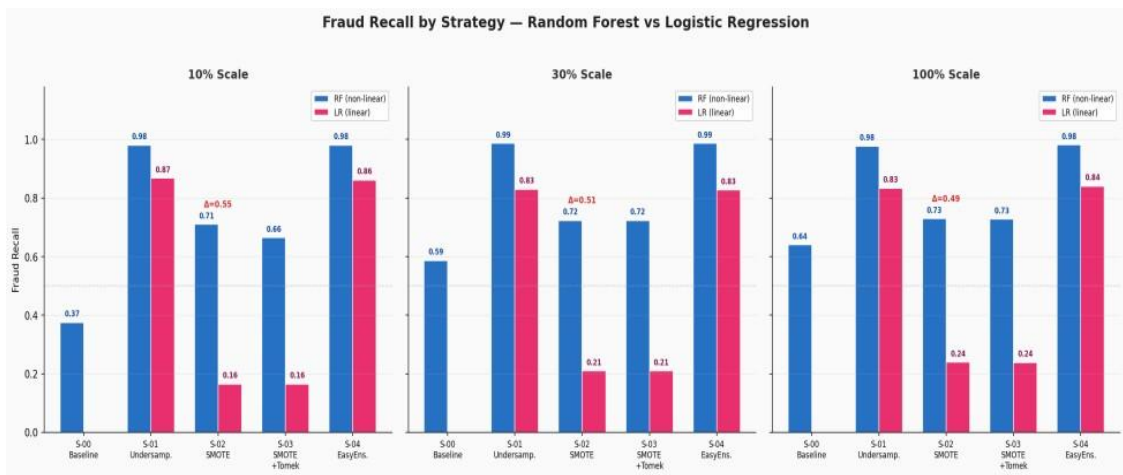


Fig. 2. Fraud Recall by strategy and scale. Blue = RF, pink = LR. Delta annotations show RF-LR gap at S-02 SMOTE: 4.3x at 10%, 3.5x at 30%, 3.1x at 100%.

Fig. 2 isolates fraud recall, the operationally critical metric. The RF-LR gap is largest at 10% scale (4.3x) and narrows slightly as scale grows, consistent with more minority training examples improving both models. However, even at 100% scale, RF SMOTE recall (0.729) remains 3.1x higher than LR SMOTE (0.239). S-01 Random Undersampling and S-04 EasyEnsemble both push RF recall above 0.97 across all scales, but at the cost of extreme false-positive rates (precision 2-3%).

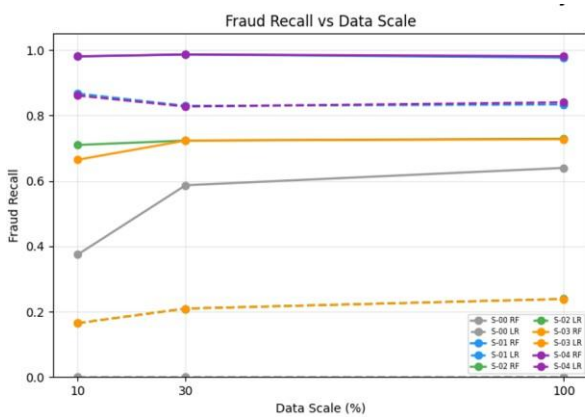


Fig. 3a — Fraud Recall vs Data Scale

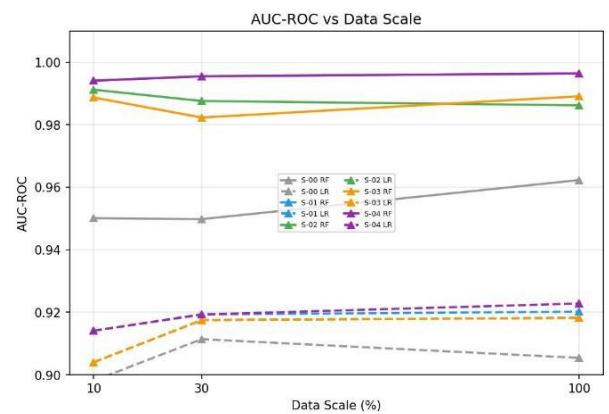


Fig. 3c — AUC-ROC vs Data Scale

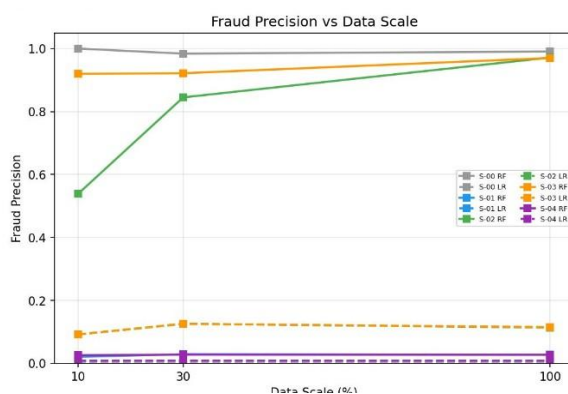


Fig. 3b — Fraud Precision vs Data Scale

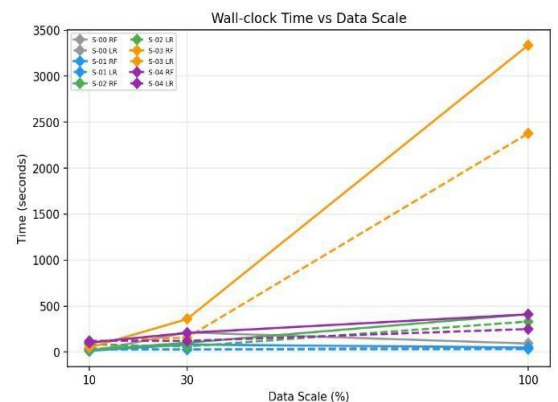


Fig. 3d — Training Time vs Data Scale

Fig. 3a–3d plot all four metrics across the five strategies and three data scales. The recall panel (Fig. 3a) shows S-01 and S-04 dominating for RF but collapsing for LR, while S-02 and S-03 exhibit a consistent RF–LR gap that widens from 10% to 100% scale. The precision panel (Fig. 3b) reveals that S-01 and S-04 achieve near-zero precision regardless of classifier, confirming that high-recall strategies are too permissive for real deployment. The AUC-ROC panel (Fig. 3c) shows all strategies except the LR baseline exceed

0.90, with RF scores uniformly higher across the board. Finally, the training-time panel (Fig. 3d) confirms that S-03 Tomek Links is the most expensive operation — 3,338 s at 100% scale due to LSH join overhead — versus 413 s for plain S-02 SMOTE. Notably, the S-02 and S-03 LR curves overlap throughout Fig. 3a–3c, reflecting that Tomek Links does not shift a linear decision boundary.

VI. CONCLUSIONS

Four distributed PySpark sampling strategies were implemented and evaluated across 30 experimental conditions on the PaySim financial fraud dataset. Key findings are:

- RF SMOTE (S-02) delivers the best practical result at full scale: Recall 0.729, Precision 0.972, F1 0.833 in 413 s.
- F1 score is the essential metric: strategies maximising Recall (S-01, S-04) achieve F1 ≈ 0.05 due to extreme false-positive rates, making them operationally unviable.
- Linear classifiers cannot exploit SMOTE: LR peaks at F1 0.154 under SMOTE versus RF's 0.833—a geometric incompatibility, not a data problem.
- SMOTE+Tomek (S-03) is not cost-effective: F1 0.831 at 8 $\times$ runtime cost versus F1 0.833 for plain SMOTE.
- All sampling is partition-local: RDD.mapPartitions with broadcast minority features and LSH approximate joins eliminate global shuffles, enabling scalability across the four-worker cluster.

Directions for future work include fine-tuning the parameters of LSH in S-03, using gradient-boosted trees (Spark MLlib GBTClassifier) as a non-linear baseline, optimization of thresholds dynamically based on each approach, and implementation on Spark Structured Streaming.

REFERENCES

- [1] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [2] I. Tomek, “Two modifications of CNN,” *IEEE Trans. Syst. Man Cybern.*, vol. SMC-6, pp. 769–772, 1976.
- [3] X.-Y. Liu, J. Wu, and Z.-H. Zhou, “Exploratory undersampling for class-imbalance learning,” *IEEE Trans. Syst. Man Cybern. B*, vol. 39, pp. 539–550, 2009.
- [4] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, “A study of the behavior of several methods for balancing machine learning training data,” *SIGKDD Explor. Newsl.*, vol. 6, no. 1, pp. 20–29, 2004.
- [5] A. Fernandez, S. Garcia, F. Herrera, and N. V. Chawla, “SMOTE for learning from imbalanced data: progress and challenges,” *J. Artif. Intell. Res.*, vol. 61, pp. 863–905, 2018.
- [6] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, “PaySim: A financial mobile money simulator for fraud detection,” in *Proc. EMSS*, 2016.
- [7] M. Zaharia et al., “Apache Spark: A unified engine for big data processing,” *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [8] L. Breiman, “Random forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [9] H. He and E. A. Garcia, “Learning from imbalanced data,” *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [10] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, “Locality-sensitive hashing scheme based on p-stable distributions,” in *Proc. SCG*, 2004, pp. 253–262.
- [11] Dataset_Link:
<https://www.kaggle.com/datasets/ealaxi/paysim1>